

Esterian Conquest

Sysop Manual

Copyright © 2026 Mason A. Green

Revision date: March 25, 2026



This manual © 2026 Mason A. Green and is licensed under CC BY-NC-SA 4.0.

License text: <https://creativecommons.org/licenses/by-nc-sa/4.0/>

Contents

1. Introduction	2
1.1. Terminology	2
2. Installation and Requirements	2
2.1. System Requirements	2
2.2. Building from Source	2
2.3. Directory Layout	3
3. Game Setup	3
3.1. Initializing a New Game	3
4. Game Directory Structure	3
4.1. Core Files	3
4.2. Subdirectories	3
5. Configuration	4
5.1. config.kdl	4
5.1.1. Full Example	4
5.1.2. Top-Level Fields	5
5.1.3. session Block	5
5.1.4. inactivity Block	5
5.1.5. reservations Block	5
5.2. Environment Variables	5
6. Theming	6
6.1. Theme File Location	6
6.2. Theme File Format	6
6.3. Color Formats	6
6.4. Color Mode	7
6.5. Semantic Style Tokens	7
6.6. ANSI Off Mode	8
7. BBS Door Setup	8
7.1. Flags for Door Mode	9
7.2. Drop File	9
7.3. Enigma BBS (Rust Client)	10
8. SSH Access	10
9. Local and Direct Play	10
10. File-Based Turn Submission	10
11. Turn Processing and Maintenance	11
12. Player Management	11
12.1. Reserving Seats	11
13. Classic Compatibility	12
14. CLI Reference	12
14.1. ec-sysop	12
14.2. ec-game	12
15. Theme Token Reference	13

1. Introduction

Esterian Conquest (EC) is a multi-player strategy game originally written for DOS-era BBS systems. The Rust-native stack reimplements the game engine, player client, and admin tooling for modern systems while preserving classic compatibility at a well-defined import/export boundary.

This manual is for the **sysop** — the person responsible for hosting a game instance. It covers:

- installing and initializing a game
- configuration and theming
- BBS door, SSH, and local deployment
- player management
- turn processing and maintenance

For player-facing rules and gameplay, see the **Player Manual**.

1.1. Terminology

game directory The directory containing `ecgame.db` and related files for a single running game instance. Passed to all tools via `--dir`.

ec-sysop The public Rust command-line sysop tool for campaign creation and maintenance.

ec-game The Rust TUI player client.

ecgame.db The SQLite database that is the runtime source of truth for the Rust engine.

themes/ Subdirectory containing theme KDL files. `themes/classic.kdl` is the default and is bootstrapped on first run. `themes/tokyo_night.kdl` is also shipped and can be activated via `config.kdl`. Sysops can add their own theme files here.

config.kdl The sysop-managed runtime configuration file in the game directory. Bootstrapped from the bundled default on first run. Controls snoop mode, session timeouts, inactivity thresholds, game name, and theme path. Changes take effect on the next `ec-game` startup.

2. Installation and Requirements

2.1. System Requirements

- Linux, macOS, or Windows
- Rust toolchain (stable) to build from source, or a pre-built binary release
- A terminal emulator for local/SSH play; a BBS server with socket support for door deployment

2.2. Building from Source

From the repository root:

```
cargo build --release -p ec-sysop -p ec-game
```

The release binaries will be in `target/release/`.

2.3. Directory Layout

A running game occupies a single directory. A minimal populated game directory looks like:

```
/path/to/mygame/
  ecgame.db           runtime database (SQLite)
  config.kdl          sysop runtime config (bootstrapped on first run)
  theme.kdl           visual theme (bootstrapped as themes/classic.kdl on first run)
  exports/            default export root for classic .DAT output
  queue/              default turn order queue directory
```

All tools take `--dir /path/to/mygame` to locate the game.

3. Game Setup

3.1. Initializing a New Game

Create a new game with `ec-sysop new-game`:

```
ec-sysop new-game /path/to/mygame --players 4 --seed 1515
```

This creates a fresh campaign directory with `ecgame.db`, classic auxiliary files, `config.kdl`, and a `themes/` subdirectory containing the bootstrapped `themes/classic.kdl` and `themes/tokyo_night.kdl` theme files.

`config.kdl` is the only sysop-edited KDL file in the public workflow. An internal `ec-cli setup-preset` format still exists for reproducible tests and harness work, but normal sysop operation does not use it.

The supported public creation flags are:

Flag	Type	Description
<code>--players</code>	integer	Number of empires. Supported range: 1–25. Defaults to 4.
<code>--seed</code>	integer	Optional campaign seed for reproducible map generation.

4. Game Directory Structure

4.1. Core Files

ecgame.db The SQLite runtime database. All game state lives here. Do not edit manually; use `ec-sysop` for normal operator actions.

themes/ Theme KDL files for `ec-game`. Bootstrapped on first run with `themes/classic.kdl` (the default) and `themes/tokyo_night.kdl`. Sysop-owned once created; not silently overwritten. See Section 6.

config.kdl Sysop runtime configuration. Bootstrapped from the bundled default on first run. Edit to change snoop mode, session timeouts, inactivity thresholds, game name, or theme path. See Section 5.

4.2. Subdirectories

exports/ Default root for classic .DAT export output. Can be overridden with `--export-root` or the `EC_CLIENT_EXPORT_ROOT` environment variable.

queue/ Default directory for turn order queue files. Can be overridden with `--queue-dir` or the `EC_CLIENT_QUEUE_DIR` environment variable.

5. Configuration

5.1. config.kdl

`config.kdl` is the sysop runtime configuration file. It lives in the game directory alongside `ecgame.db`. If absent when `ec-game` starts, it is bootstrapped from the bundled default automatically.

Changes to `config.kdl` take effect on the next `ec-game` startup. The runtime policy settings backed by `SETUP.DAT` bytes are applied into the runtime database at that point; no manual database edits are required.

5.1.1. Full Example

```
// Display name shown in the main menu header.
game_name "Esterian Conquest"

// Theme file (relative to game directory, or absolute path).
// Shipped themes: themes/classic.kdl (default), themes/tokyo_night.kdl
// Omit to use themes/classic.kdl.
// theme "themes/classic.kdl"

// Sysop snoop: set to #false to disable.
snoop #true

// Session timeout and timing policies.
session {
    // Minutes of inactivity before timeout (0-120).
    max_idle_minutes 10
    // Minimum time granted per session in minutes (0-120).
    minimum_time_minutes 0
    // Apply timeout to local (non-remote) sessions.
    local_timeout #false
    // Apply timeout to remote sessions.
    remote_timeout #true
}

// Inactivity thresholds (in turns). Set to 0 to disable.
inactivity {
    // Purge a player after this many inactive turns (0-100).
    purge_after_turns 0
    // Put a player on autopilot after this many inactive turns (0-100).
    autopilot_after_turns 0
}

// Optional BBS/dropfile seat reservations by caller alias.
reservations {
    seat player=1 alias="SYSOP"
    seat player=2 alias="NightShade"
}
```

5.1.2. Top-Level Fields

Field	Type	Default	Description
game_name	string	"Esterian Conquest"	Display name shown in the main menu header.
theme	string	<i>(absent)</i>	Theme file path, relative to the game directory. Omit to use themes/classic.kdl. Example: "themes/tokyo_night.kdl".
snoop	bool	#true	Enable sysop snoop mode.
reservations	block	<i>(absent)</i>	Optional BBS/dropfile seat reservations by caller alias.

5.1.3. session Block

Field	Type	Default	Description
max_idle_minutes	integer	10	Minutes of inactivity before session timeout. Range: 0–120.
minimum_time_minutes	integer	0	Minimum session time guarantee in minutes. Range: 0–120.
local_timeout	bool	#false	Apply timeout to local (non-remote) sessions.
remote_timeout	bool	#true	Apply timeout to remote sessions.

5.1.4. inactivity Block

Field	Type	Default	Description
purge_after_turns	integer	0	Turns of inactivity before a player is purged. 0 = disabled. Range: 0–100.
autopilot_after_turns	integer	0	Turns of inactivity before autopilot engages. 0 = disabled. Range: 0–100.

5.1.5. reservations Block

Field	Type	Default	Description
seat player=<N> alias="NAME"	entry	<i>(absent)</i>	Reserve empire slot <i>n</i> for a BBS/dropfile caller alias. Alias matching is ASCII case-insensitive.

NOTE: config.kdl is for sysop use only. Players do not interact with it. Fields not present in the file use their default values. Omitting a field is equivalent to setting it to its default.

5.2. Environment Variables

Variable	Description
----------	-------------

EC_CLIENT_EXPORT_ROOT	Override the default export root directory.
EC_CLIENT_QUEUE_DIR	Override the default turn queue directory.
COLORTERM	Color depth hint. <code>truecolor</code> or <code>24bit</code> enables 24-bit color.
TERM	Terminal type. A value containing <code>256color</code> enables 256-color mode.

6. Theming

ec-game uses a file-driven theme system. The theme defines the visual appearance for all players connecting to this game directory.

6.1. Theme File Location

ec-game resolves the theme in this order:

1. If `<game_dir>/config.kdl` contains a theme directive, use that path (relative to `game_dir`).
2. Otherwise, use `<game_dir>/themes/classic.kdl`.
3. If `themes/classic.kdl` does not exist, create the `themes/` directory and bootstrap it from the bundled default.

`config.kdl` itself is bootstrapped on first run if absent, so it is always present by the time this resolution runs. The `theme` directive within it is optional; omitting it falls through to step 2.

On parse error, the bundled default is used silently so a corrupted theme never prevents players from connecting.

6.2. Theme File Format

A theme file is a KDL document. Each visual element is declared as a `style` node with a name and child `fg`, `bg`, and optional `bold` fields.

```
style "body" {
  fg "white"
  bg "black"
}

style "logo" {
  fg "bright_blue"
  bg "black"
  bold #true
}

style "selected" {
  fg "black"
  bg "bright_blue"
}
```

The star decoration colors are declared separately:

```
star-colors "bright_blue" "bright_white" "white" "bright_yellow" "yellow" "bright_red"
```

6.3. Color Formats

Three color formats are supported:

Format	Example	Notes
Named ANSI-16	"bright_blue"	Safe for all terminals including BBS/door clients. Recommended default.
256-color index	"idx:208"	Requires <code>--color-mode 256</code> or <code>truecolor</code> . Downgraded to nearest named color in 16-color mode.
24-bit hex RGB	"#ff8800"	Requires <code>--color-mode truecolor</code> . Downgraded gracefully in lower color modes.

Two themes ship with `ec-game`. `themes/classic.kdl` is the default: a restrained dark palette using named ANSI-16 colors only, safe for all terminal types including BBS door clients. `themes/tokyo_night.kdl` uses a full 24-bit hex palette (deep navy background with purple, teal, and amber accents) and is intended for modern local or SSH terminals. Both degrade gracefully — all hex colors are automatically mapped to the nearest 256-color index or 16-color name on terminals that do not support `truecolor`. Custom themes may use any of the three color formats.

6.4. Color Mode

`ec-game` selects a color mode at startup:

Mode	Description
<code>ansi16</code>	Classic 16-color ANSI. Safe for BBS/door and all terminal types.
<code>256</code>	256-color xterm palette.
<code>truecolor</code>	24-bit RGB. Best for modern local or SSH terminals.
<code>auto</code>	Detected from <code>COLORTERM</code> and <code>TERM</code> environment variables.

Override with the `--color-mode` flag:

```
ec-game --dir /path/to/mygame --player 1 --color-mode truecolor
```

When `--encoding cp437` is specified (BBS door mode), `ec-game` defaults to `ansi16` regardless of the environment, unless `--color-mode` is set explicitly.

6.5. Semantic Style Tokens

All required style tokens:

Token	Purpose
<code>body</code>	Default body text.
<code>title</code>	Screen and section titles.
<code>menu</code>	Menu item text.
<code>menu_hotkey</code>	Menu hotkey letters.
<code>prompt</code>	Input prompt text.
<code>prompt_hotkey</code>	Prompt hotkey letters.
<code>prompt_notice_action</code>	Action-notice accent in prompts.

bright	Emphasized body text.
logo	Title logo decoration.
intro_accent	Intro screen accent color.
intro_tribute	Intro screen tribute line.
stardate_label	Stardate label.
stardate_week	Stardate week value.
stardate_year	Stardate year value.
error	Error messages.
notice	Warning / notice messages.
status_label	Status bar labels.
status_value	Status bar values.
table_chrome	Table borders and separators.
table_header	Table column headers.
table_body	Table body rows.
disabled_row	Disabled / unavailable table rows.
selected	Selected / highlighted row.
alert	High-priority alert text.
help_header	Help overlay section headers.
help_panel	Help overlay body text.
map_dot	Star map background dots.
map_crosshair	Star map crosshair / cursor.
map_center	Star map center marker.
quote	Quote / flavor text body.
quote_author	Quote attribution.
report_header	Report page headers.
indicator_on	Active indicator (e.g. status lit).
indicator_off	Inactive indicator.

6.6. ANSI Off Mode

Players can toggle ANSI color off within a session (a session-level preference only — it does not modify theme files on disk). ANSI Off applies a monochrome projection over the loaded theme: white/black with preserved reverse-video selection. A new session always starts with ANSI On.

7. BBS Door Setup

ec-game can run as a BBS door. In door mode it reads from and writes to a socket instead of the local TTY, using CP437 encoding and 16-color ANSI.

7.1. Flags for Door Mode

```
ec-game \  
  --dir /path/to/mygame \  
  --player <1-based index> \  
  --encoding cp437 \  
  --color-mode ansi16
```

--encoding cp437 switches box-drawing characters and other extended characters to the CP437 code page, which is required for correct rendering in classic ANSI-aware BBS terminals (SyncTERM, NetRunner, etc.).

When --encoding cp437 is active, --color-mode defaults to ansi16 automatically. Override only if you know your BBS clients support a richer color depth.

7.2. Drop File

ec-game can read a BBS drop file directly with --dropfile <path>:

```
ec-game \  
  --dir /path/to/mygame \  
  --dropfile /path/to/D00R32.SYS
```

Supported drop file formats (auto-detected by filename, case-insensitive):

- D00R32.SYS — modern standard (Enigma, Mystic, Talisman, etc.)
- D00R.SYS — legacy, widest BBS software support
- CHAIN.TXT — WWIV format

The drop file supplies the player alias and session time limit. Explicit CLI flags always override drop file values. When --dropfile is given and --encoding is not, encoding defaults to cp437 automatically.

--timeout <minutes> sets a session time limit independently of a drop file.

Reserve seats in config.kd1 when you want the caller alias to determine the empire automatically:

```
reservations {  
  seat player=1 alias="SYSOP"  
  seat player=2 alias="NightShade"  
}
```

With that in place, a reserved caller can launch with --dropfile alone. If the caller alias is not reserved, --player is still required. If both --player and --dropfile are supplied for a reserved caller, they must agree on the same empire slot.

NOTE: The original DOS ECGAME.EXE v1.5 expects a strict 32-line WWIV-style CHAIN.TXT drop file. Enigma BBS-generated D00R.SYS / DORINFO files will crash ECGAME.EXE. See docs/sysop/enigma-bbs-setup.md for the full legacy DOS door path if you need to host the original binary.

7.3. Enigma BBS (Rust Client)

To run the native `ec-game` as an Enigma BBS door, use the `abracadabra` module with `io: socket` and pass `--dir`, `--encoding cp437`, and `--color-mode ansi16` as arguments. Use `--player` for unreserved callers, or reserve the alias in `config.kdl` and let `--dropfile` resolve the seat. The client will inherit the socket from Enigma's stdio handoff.

If Enigma writes a `D00R32.SYS`, you can pass it directly:

```
ec-game \  
  --dir /path/to/mygame \  
  --dropfile /path/to/D00R32.SYS
```

8. SSH Access

`ec-game` runs cleanly over SSH. No special flags are required for SSH sessions on modern terminals.

Color mode is auto-detected from the environment:

- `COLORTERM=truecolor` → 24-bit RGB
- `TERM` containing `256color` → 256-color
- Otherwise → 16-color ANSI fallback

Force a specific mode with `--color-mode` if your SSH setup does not propagate `COLORTERM` reliably:

```
ec-game --dir /path/to/mygame --player 1 --color-mode 256
```

UTF-8 encoding (the default) is correct for SSH sessions on modern terminals. Use `--encoding cp437` only if you are proxying through a BBS or a CP437 terminal emulator over SSH.

9. Local and Direct Play

For localhost play, run `ec-game` directly in your terminal:

```
ec-game --dir /path/to/mygame --player 1
```

Color mode and encoding default to `auto / utf8`, which is correct for modern terminal emulators. The client detects `COLORTERM` and `TERM` automatically.

10. File-Based Turn Submission

Players on localhost or a shared host can use either the interactive TUI or `ec-game submit-turn`.

```
ec-game submit-turn --check --dir /path/to/mygame --player 1 --file /path/to/player1-turn.kdl  
ec-game submit-turn --dir /path/to/mygame --player 1 --file /path/to/player1-turn.kdl
```

This is a supported file-based interface for manual workflows and custom client integration. It writes directly to `ecgame.db` after the submission validates cleanly. If any command in the file is invalid, the entire submission is rejected and nothing is written.

Do not treat this as a queue processor or upload inbox. `submit-turn` is a direct command that reads one KDL file and applies it immediately to the live campaign state.

11. Turn Processing and Maintenance

Run yearly maintenance with:

```
ec-sysop maint /path/to/mygame [turns]
```

`ec-sysop maint` advances the campaign in `ecgame.db`. EC does not schedule maintenance by itself. In a real deployment, invoke `ec-sysop maint` from your host scheduler or BBS tooling:

- a systemd timer
- cron
- a BBS event runner
- or manual sysop operation

Do not treat KDL config as a scheduler. `config.kdl` owns runtime policy such as theming, snoop, and inactivity thresholds. Maintenance timing belongs to the host.

12. Player Management

Inactive-player policy is configured in `config.kdl` under the `inactivity` block. The two public thresholds are:

- `purge_after_turns`
- `autopilot_after_turns`

These values are runtime policy, not setup-time game creation input.

12.1. Reserving Seats

To reserve empire slots for specific BBS users, add a `reservations` block to `config.kdl`:

```
reservations {  
    seat player=1 alias="SYSOP"  
    seat player=2 alias="NightShade"  
}
```

Each `seat` entry binds one 1-based empire slot to one caller alias. Alias matching is ASCII case-insensitive, so `NightShade`, `nightshade`, and `NIGHTSHADE` are treated as the same reservation.

With reservations in place, launch `ec-game` with `--dropfile` and let the caller alias choose the empire automatically:

```
ec-game --dir /path/to/mygame --dropfile /path/to/D00R32.SYS
```

Important rules:

- if the caller alias is reserved, `--player` becomes optional
- if the caller alias is not reserved, `--player` is still required
- if both `--player` and `--dropfile` are supplied for a reserved caller, they must match
- if a reservation conflicts with an already-stored different player handle, `ec-game` will stop with a clear error so the sysop can reconcile `config.kdl` and the runtime state

Reserving a seat does not by itself join or pre-name the empire. It only routes the caller to the intended slot. The usual first-time join flow still claims an open empire, and that first successful join records the caller alias into the player record for later logins.

13. Classic Compatibility

The Rust-native public deployment path is `ec-sysop` plus `ec-game`.

Classic .DAT import/export, oracle runs against the original binaries, and other preservation workflows still exist, but they belong to the internal `ec-cli` developer/compatibility surface rather than the normal `sysop` manual.

14. CLI Reference

14.1. ec-sysop

`ec-sysop <subcommand> [options]`

Subcommand	Purpose
<code>new-game</code>	Create a fresh campaign directory. Public flags: <code>--players</code> and <code>--seed</code> .
<code>maint</code>	Run one or more maintenance turns against <code>ecgame.db</code> .

14.2. ec-game

`ec-game --dir <game_dir> [--player <1-based index>] [options]`

`ec-game submit-turn [--check] --dir <game_dir> --player <record> --file <turn.kdl>`

Interactive client flags:

Flag	Description
<code>--dir <path></code>	Game directory containing <code>ecgame.db</code> . Required.
<code>--player <N></code>	1-based empire index. Required unless a reserved dropfile alias resolves the seat from <code>config.kdl</code> .
<code>--encoding <utf8 cp437></code>	Output encoding. Default: <code>utf8</code> . Use <code>cp437</code> for BBS/door mode.
<code>--color-mode <ansi16 256 truecolor auto></code>	Color depth. Default: <code>auto</code> (env-detected). CP437 mode defaults to <code>ansi16</code> .
<code>--dropfile <path></code>	Parse a BBS drop file (<code>DOOR32.SYS</code> , <code>DOOR.SYS</code> , or <code>CHAIN.TXT</code>). Supplies alias and timeout, defaults encoding to <code>cp437</code> , and can resolve the player seat through <code>config.kdl</code> reservations. Explicit flags always override except that <code>--player</code> must match a reserved alias when both are present.

--timeout <minutes>	Session time limit in minutes. Overrides any drop file value.
--export-root <path>	Override export directory. Default: <game_dir>/exports.
--queue-dir <path>	Override turn queue directory. Default: <game_dir>/queue.

submit-turn flags:

Flag	Description
--check	Validate the KDL file without mutating the campaign.
--dir <path>	Game directory containing ecgame.db. Required.
--player <N>	1-based empire index. Required, and must match the KDL header.
--file <path>	Turn submission KDL file to validate or apply. Required.

NOTE: ec-game submit-turn is all-or-nothing. If any command in the file is invalid, the entire submission is rejected and nothing is written to ecgame.db.

15. Theme Token Reference

The default bundled theme values are listed below for reference. All colors use named ANSI-16 values.

Token	fg	bg	bold
body	white	black	—
title	bright_blue	black	yes
menu	white	black	—
menu_hotkey	yellow	black	yes
prompt	white	black	—
prompt_hotkey	yellow	black	yes
prompt_notice_action	bright_cyan	black	yes
bright	bright_white	black	yes
logo	bright_blue	black	yes
intro_accent	bright_blue	black	—
intro_tribute	bright_magenta	black	—
stardate_label	cyan	black	—
stardate_week	bright_cyan	black	—
stardate_year	yellow	black	—
error	red	black	yes

notice	bright_red	black	yes
status_label	white	black	—
status_value	white	black	—
table_chrome	blue	black	—
table_header	cyan	black	yes
table_body	bright_white	black	—
disabled_row	bright_black	black	—
selected	black	bright_blue	—
alert	bright_white	red	yes
help_header	bright_blue	black	yes
help_panel	white	black	—
map_dot	green	black	—
map_crosshair	bright_red	black	yes
map_center	bright_white	black	yes
quote	white	black	—
quote_author	white	black	—
report_header	cyan	black	—
indicator_on	bright_green	black	yes
indicator_off	bright_black	black	—

Star decoration colors (6 slots, cycling):

star-colors "bright_blue" "bright_white" "white" "bright_yellow" "yellow" "bright_red"